

TECH

REACT CORE

RENDERING HOOKS **EFFECTS** REACT 18+ / CONCURRENCY

EFFECTS

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК react.dev

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	ЗАЧЕМ useEffect UNCHRONIZE Q1 · Q2 · Q3	КОГДА НЕ НУЖЕН	S	Эффект — это синхронизация с внешним миром, не «делай что-то».
ДЕНЬ 2 ЛЁГКИЙ	DERIVED STATE — АНТИПАТТЕРН Q4			Один из самых частых антипаттернов и его лечение.
ДЕНЬ 3 ТЯЖЁЛЫЙ	DEPENDENCIES ЗНАННОЕ УБРАТЬ Q5 · Q6 · Q7 · Q8	EXHAUSTIVE-DEPS	OCO	Главная боль useEffect — правильные зависимости.
ДЕНЬ 4 СРЕДНИЙ	INFINITE LOOP Q9 · Q10	CLEANUP		Как не зациклиться и правильно убирать ресурсы.
ДЕНЬ 5 ТЯЖЁЛЫЙ	ASYNC В EFFECT Q11 · Q12	RACE CONDITION		Аsync и гонки запросов в эффектах.
ДЕНЬ 6 СРЕДНИЙ	useLayoutEffect POTENT Q13 · Q14 · Q15 · Q16 · Q17	STRICT MODE	IDEM	Слой layout-эффектов и устойчивость к двойному запуску.
ДЕНЬ 7 СРЕДНИЙ	SUBSCRIPTIONS RS Q18 · Q19 · Q20	TIMERS	DOM LISTENERS	Три классических ресурса в effect — корректно подписаться и снять.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

ЗАЧЕМ `useEffect` КОГДА НЕ НУЖЕН `SYNCHRONIZE`

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Эффект — это синхронизация с внешним миром, не «делай что-то».

ВОПРОСЫ ДНЯ

- Q01 Для чего нужен `useEffect`?
- Q02 Когда `useEffect` не нужен?
- Q03 Что значит "synchronize with external system"?

Для чего нужен useEffect?

ОПИСАНИЕ

`useEffect` — синхронизирует React-компонент с внешним миром: сетью, таймерами, подписками, DOM-узлами вне React, аналитикой. Эффект запускается ПОСЛЕ `commit`, когда DOM уже обновлён. Это правильное место для побочных действий: всё, что не должно идти в `render`.

КАК РАБОТАЕТ

- 01 `useEffect(fn, deps)` — `fn` запускается после `commit`.
- 02 Cleanup-функция в `return` — выполняется перед следующим запуском или при `unmount`.
- 03 `deps` определяют, нужно ли перезапустить.
- 04 Без `deps` (без массива) — каждый рендер. С `[]` — один раз. С `[a, b]` — при смене.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Загрузка данных по `props (id)`.
- 02 Подписка на `WebSocket / EventEmitter`.
- 03 `setInterval / setTimeout`.
- 04 DOM-listeners на `window/document`.
- 05 Логирование, аналитика.

ПОДВОДНЫЕ КАМНИ

- 01 Часто пишут эффекты для `derived state` — это антипаттерн (см. Q4).
- 02 Эффект без `cleanup` → утечки и `race`.
- 03 В `Strict Mode dev` — двойной запуск (см. Q15).
- 04 `useEffect` не подходит для вычислений — это для побочных эффектов.

КОД

// Базовое

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/u/${id}`, { signal: ctrl.signal })
    .then(r => r.json()).then(setUser)
    .catch(e => { if (e.name !== 'AbortError') throw e });
  return () => ctrl.abort();
}, [id]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `useEffect` отличается от `useLayoutEffect`?
- 02 Когда не нужен `useEffect`?

03 Что значит «синхронизация с внешним миром»?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

Когда useEffect не нужен?

ОПИСАНИЕ

useEffect не нужен для вычислений на основе props/state (derived state – считать прямо в render); для логики обработчиков (вызывать функцию по клику, не через effect+флаг); для трансформации данных перед показом (useMemo). Если работа не «синхронизация с внешним миром» – почти всегда есть способ без useEffect.

КАК РАБОТАЕТ

- 01 Derived value: `const fullName = first + ' ' + last` – просто переменная.
- 02 Тяжёлое derived: `useMemo`, не `useEffect`.
- 03 Действие по клику: обработчик, не `effect`.
- 04 Сброс state при смене prop: `key={prop}` вместо `useEffect + setState`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Refactor «strange useEffect» – переписать как мемо / обработчик / `key`.
- 02 Обучение junior'ов.
- 03 Code-review – отлавливать «You Might Not Need an Effect».

ПОДВОДНЫЕ КАМНИ

- 01 Эффект, считающий derived state и кладущий его в `useState` → лишний `render`.
- 02 Эффект, реагирующий на `click` → лучше обработчик.
- 03 Эффект, делающий `push` в `roots` на каждый рендер.

КОД

// Антипаттерн

```
useEffect(() => { setFull(first + ' ' + last); }, [first, last]);
```

// Правильно

```
const full = first + ' ' + last;
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие три классических случая «эффект не нужен»?
- 02 Чем `key`-сброс лучше эффекта-сброса?
- 03 Когда обработчик лучше эффекта?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/you-might-not-need-an-effect>

ВОПРОС Q03

Что значит "synchronize with external system"?

ОПИСАНИЕ

Внешняя система – всё, что не управляется React: сеть, локальное хранилище, DOM-API не через JSX, сторонние библиотеки (D3, видеоплееры), таймеры, Web APIs. Эффект – двусторонний мост между React-state и таким миром: на mount/смену deps мы «настраиваем» внешнюю систему под текущий state, в cleanup – разбираем.

КАК РАБОТАЕТ

- 01 На запуске эффекта – setup: подписаться, создать ресурс, запросить данные.
- 02 В cleanup – teardown: отписаться, отменить, освободить.
- 03 При смене deps – cleanup старой версии + setup новой.
- 04 Это и есть «синхронизация»: внешний мир следует за React-state.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 WebSocket/EventEmitter подписки.
- 02 Сторонние UI-либы (Mapbox, Three.js).
- 03 Сетевые запросы по props.
- 04 DOM-listeners.

ПОДВОДНЫЕ КАМНИ

- 01 Setup без teardown → утечки.
- 02 Teardown без понимания → отключаем то, что нужно.
- 03 Внешний мир иногда асинхронный – нужны race-protections.

КОД

// Подписка

```
useEffect(() => {
  const sub = source.subscribe(setData);
  return () => sub.unsubscribe();
}, [source]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое «внешняя система» в React-парадигме?
- 02 Почему cleanup – обязательная часть синхронизации?
- 03 Чем отличается setup-teardown от mount-unmount?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

DERIVED STATE – АНТИПАТТЕРН

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Один из самых частых антипаттернов и его лечение.

ВОПРОСЫ ДНЯ

Q04 Почему derived state через effect часто плохой паттерн?

ВОПРОС Q04

Почему derived state через effect часто плохой паттерн?

ОПИСАНИЕ

Derived state – то, что вычисляется из других значений (props/state). Делать это через `useEffect` + `setState` даёт лишний `rerender` (сначала рендер с устаревшим, затем эффект поставит новое, ещё рендер), может породить бесконечный цикл, усложняет дебаг (state живёт в нескольких местах). Решение: считать прямо в `render`, или `useMemo` для тяжёлых.

КАК РАБОТАЕТ

- 01 `Render` → старое `derived` → `setState` в `effect` → второй `render` → новое `derived`.
- 02 Useless `if` – двойной `render`.
- 03 Дешёвое `derived` – просто переменная.
- 04 Дорогое – `useMemo`.
- 05 Иногда (редко) `derived` должно сохраняться при сбросе родительского `prop` – тогда оставить `state`, но сбрасывать через `key`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аудит компонентов на лишние эффекты.
- 02 Преобразование форм-`state`.
- 03 Производный список (фильтрованный, отсортированный).

ПОДВОДНЫЕ КАМНИ

- 01 Бесконечный цикл при неправильных `deps`.
- 02 Дёрганный UI на этапе "первый рендер с пустым".
- 03 Тестирование сложнее.
- 04 Чтобы починить – иногда нужен рефакторинг архитектуры.

КОД

// Плохо

```
useEffect(() => {
  setSorted(items.slice().sort((a, b) => a.x - b.x));
}, [items]);
```

// Хорошо

```
const sorted = useMemo(
  () => items.slice().sort((a, b) => a.x - b.x),
  [items]
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Сколько ререндеров вызывает `derived` через эффект?

- 02 Когда useМемо не нужен и достаточно переменной?
- 03 Как сбросить state при смене проп без эффекта?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/you-might-not-need-an-effect>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

DEPENDENCIES**EXHAUSTIVE-DEPS****ОСОЗНАННОЕ УБРАТЬ****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Главная боль `useEffect` — правильные зависимости.

ВОПРОСЫ ДНЯ

- Q05 Как правильно указывать `dependencies`?
- Q06 Почему ESLint `exhaustive-deps` важен?
- Q07 Когда можно осознанно убрать `dependency`?
- Q08 Что делать, если `dependency` постоянно меняется?

Как правильно указывать dependencies?

ОПИСАНИЕ

Правильно — указать ВСЕ значения из тела эффекта, которые приходят из render-области (props, state, переменные из замыкания). React сравнивает их через Object.is. Если что-то не указано — будет stale closure. Если что-то лишнее — ненужные перезапуски. Линтер react-hooks/exhaustive-deps делает 99% работы.

КАК РАБОТАЕТ

- 01 Все props/state/локальные переменные из тела — в deps.
- 02 Object.is-сравнение — нужны стабильные ссылки для объектов/функций.
- 03 useCallback/useMemo помогают стабилизировать.
- 04 Reactive values — то, что меняется при render.
- 05 Стабильные ref-объекты, dispatch, set-функции из useState — НЕ нужно класть в deps.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой useEffect/useCallback/useMemo.
- 02 При линтер-варнинге — добавить или подумать, не лишний ли effect.

ПОДВОДНЫЕ КАМНИ

- 01 Игнорирование eslint-warning → stale closures.
- 02 Объекты в deps без useMemo → бесконечный цикл.
- 03 Функции в deps без useCallback → то же.
- 04 Лишний ref в deps — не нужен.

КОД

// Полный список

```
useEffect(() => {
  fetchUser(id, options).then(setUser);
}, [id, options]); // options тоже
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие значения НЕ нужно класть в deps?
- 02 Чем helps useCallback в deps?
- 03 Когда добавление deps создаёт цикл?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect#specifying-reactive-dependencies>

ВОПРОС Q06

Почему ESLint exhaustive-deps важен?

ОПИСАНИЕ

exhaustive-deps – линтер из eslint-plugin-react-hooks. Проверяет, что в deps указаны ВСЕ reactive-зависимости из тела хука. Игнорирование почти всегда ведёт к stale closures и тонким багам. Это один из лучших инструментов качества React-кода. Отключение – почти всегда симптом архитектурной проблемы.

КАК РАБОТАЕТ

- 01 Анализирует AST: какие переменные читает хук-колбэк.
- 02 Сравнивает с указанными deps.
- 03 Ругается на missing и extra.
- 04 Ловит распространённые ошибки: stale, infinite loop при неправильных deps.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой useEffect/useCallback/useMemo.
- 02 CI с обязательным lint-passing.
- 03 Pre-commit hook.

ПОДВОДНЫЕ КАМНИ

- 01 // eslint-disable-next-line react-hooks/exhaustive-deps – должно быть редкостью, с обоснованием.
- 02 Линтер не понимает динамические deps (computed keys) – иногда false positive.
- 03 Большой список deps часто признак, что нужно разделить эффект.

КОД

// Linter ловит

```
useEffect(() => { fetchUser(id) }, []);  
// warning: 'id' is missing
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делать с false positive?
- 02 Можно ли отключить линтер для одной строки?
- 03 Чем большой список deps – bad smell?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

Когда можно осознанно убрать dependency?

ОПИСАНИЕ

Только когда есть явная причина и понимание risks. Кейсы: ссылка стабильна по жизни компонента (set-функция `useState`, `dispatch`, `ref`); значение не должно перезапускать эффект (используем `ref` + `sync-effect` для свежей ссылки); значение реально не зависит от значения, а только от идентичности (например, `count` для `on-mount`-аналитики).

КАК РАБОТАЕТ

- 01 `set`-функции `useState` – стабильны, не нужны в `deps`.
- 02 `useReducer` `dispatch` – стабилен.
- 03 `useRef`-объект – стабилен.
- 04 Если значение нужно «свежим, но не триггером» – кладем в `ref` через `sync-effect`.
- 05 Перед отключением `eslint` – комментарий с обоснованием.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аналитика «один раз на `mount`» – `useEffect(...)`.
- 02 `useEvent-pattern` с `ref` для стабильного хендлера.
- 03 Защитая стабильность (`immutable` константы).

ПОДВОДНЫЕ КАМНИ

- 01 Ошибка часто маскирует реальную проблему – лучше переписать.
- 02 Без комментария будущему программисту неясно почему.
- 03 `Ref-pattern` в эффекте, который должен пере-запускаться – ломает синхронизацию.

КОД

// Уселинтер с комментарием

```
useEffect(() => {  
  // mount-only analytics  
  // eslint-disable-next-line react-hooks/exhaustive-deps  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие зависимости стабильны автоматически?
- 02 Чем `useEvent` помогает?
- 03 Когда `ref-pattern` уместен?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/separating-events-from-effects>

ВОПРОС Q08

Что делать, если dependency постоянно меняется?

ОПИСАНИЕ

Если `dep` пересоздаётся каждый рендер (объект, функция), эффект запускается без причины. Решения: стабилизировать ссылку (`useMemo` для объектов, `useCallback` для функций); вынести объект из `render` (если он константа); поднять его в `context/store`, где он живёт дольше; поменять архитектуру эффекта (возможно, ему не нужно эту зависимость отслеживать).

КАК РАБОТАЕТ

- 01 Inline-объект → `useMemo({ ... }, [primitive deps])`.
- 02 Inline-функция → `useCallback(fn, deps)`.
- 03 Константа → вынести из компонента.
- 04 Структурно-нестабильная (json from props) → `useMemo` с прим-`deps` (`id`) или сериализация ключа.
- 05 Иногда правильнее переключить эффект на `ref-pattern`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Эффект бьётся в `loop` из-за объекта в `props`.
- 02 `useEffect` с `config`-объектом.
- 03 API-параметры, генерируемые в `render`.

ПОДВОДНЫЕ КАМНИ

- 01 `useMemo` без правильных `deps` снова даст нестабильную ссылку.
- 02 Глубокое сравнение – медленно и не нужно.
- 03 Слишком мемоизированный код перестаёт быть читаемым.

КОД

// Стабилизация

```
const opts = useMemo(() => ({ headers: { auth } }), [auth]);
useEffect(() => fetchData(opts), [opts]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `useMemo` помогает, а когда нет?
- 02 Что делать, если `deps` пришли из `props`?
- 03 Чем плох глубокий `compare`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect#removing-unnecessary-object-dependencies>

INFINITE LOOP

CLEANUP

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как не зациклиться и правильно убирать ресурсы.

ВОПРОСЫ ДНЯ

Q09 Как избежать infinite effect loop?

Q10 Как правильно делать cleanup?

ВОПРОС Q09

Как избежать infinite effect loop?

ОПИСАНИЕ

Loop возникает, когда effect меняет state, которое находится в его deps – каждое изменение запускает эффект заново. Лечение: проверить, действительно ли нужно зависеть от этого state; убрать derived через effect (см. Q4); использовать functional update `setN(prev => ...)`; вынести из deps стабилизированной ссылкой.

КАК РАБОТАЕТ

- 01 Effect → setState → новый render → effect снова – loop.
- 02 Лечение 1: condition на setState (не каждый раз).
- 03 Лечение 2: убрать dep на state, использовать functional update.
- 04 Лечение 3: убрать сам эффект – это derived, считать в render.
- 05 Лечение 4: ref-pattern для свежего значения без deps.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Эффекты с setState внутри.
- 02 Эффекты по props, которые тоже меняются от state.
- 03 Reducer-связки, где dispatch триггерит state, триггерит effect.

ПОДВОДНЫЕ КАМНИ

- 01 Дев-сервер начинает гореть – горячий браузер.
- 02 Profiler показывает миллисекундные ререндеры в цикле.
- 03 Иногда loop виден только при определённых условиях ввода.

КОД

// Loop

```
useEffect(() => {  
  setN(n + 1); // меняет n, dep на n → loop  
}, [n]);
```

// Не loop

```
useEffect(() => {  
  setN(prev => prev + 1); // dep пустой  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Как заметить loop в DevTools?
- 02 Когда нужен setState в effect, а когда нет?
- 03 Чем лечит functional update?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

ВОПРОС Q10

Как правильно делать cleanup?

ОПИСАНИЕ

Cleanup — функция, возвращаемая из effect-колбэка. Она вызывается перед следующим запуском effect (при смене deps) и при размонтировании. Цель — отменить всё, что было настроено в setup: подписки, таймеры, запросы, DOM-listeners. Корректный cleanup защищает от утечек памяти и race conditions.

КАК РАБОТАЕТ

- 01 return () => { ... } из callback useEffect.
- 02 Вызывается перед следующим setup и на unmount.
- 03 В Strict Mode dev — на каждом mount-unmount-mount цикле.
- 04 Должен быть симметричен setup-у: подписался → отписался; setTimeout → clearTimeout.
- 05 Не должен зависеть от внешнего state — захватывать ресурсы в локальные переменные.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой эффект с setup-ресурсом.
- 02 Подписки, таймеры, fetch, DOM-listeners.
- 03 Strict Mode dev — обязательно.

ПОДВОДНЫЕ КАМНИ

- 01 Забытый cleanup → утечки и stale UI.
- 02 Cleanup, обращающийся к свежему state — не сработает (он замкнут на старое).
- 03 Async-cleanup невозможен напрямую — нужно synchronous-обёртка.
- 04 Двойной cleanup (Strict Mode) — должна быть идемпотентность (см. Q17).

КОД

```
// Симметричный setup-teardown
```

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id);  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда срабатывает cleanup?
- 02 Можно ли вернуть async-функцию?
- 03 Что делает Strict Mode с cleanup?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

ASYNC В EFFECT RACE CONDITION

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Async и гонки запросов в эффектах.

ВОПРОСЫ ДНЯ

Q11 Как отменять async work в effect?

Q12 Как избежать race condition в effect?

Как отменять async work в effect?

ОПИСАНИЕ

Аsync-работа в effect – fetch, чтение потока, тяжёлые вычисления через Promise – должна уметь отменяться. Стандарт: AbortController, signal в fetch, abort в cleanup. Альтернатива: cancelled-флаг в замыкании. Без отмены – race condition: ответ старого запроса перезапишет свежий (см. Q12).

КАК РАБОТАЕТ

- 01 AbortController – стандарт. Создаём → передаём signal → abort в cleanup.
- 02 fetch с aborted signal реджектится с AbortError, нужно отдельно фильтровать.
- 03 cancelled-флаг – для не-fetch аsync-работы.
- 04 Аsync-функция объявлена внутри effect, вызвана сразу.
- 05 В react-query / SWR это автоматизировано.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Загрузка данных по props.
- 02 Поиск с автоподсказками.
- 03 Аsync-операции, которые могут устареть.

ПОДВОДНЫЕ КАМНИ

- 01 AbortError в общем catch-блоке маскирует ошибку.
- 02 useEffect не должен возвращать promise – аsync внутри.
- 03 Без abort – stale response (см. Q12).
- 04 Отмена fetch не отменяет работу на сервере.

КОД

// AbortController

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/u/${id}`, { signal: ctrl.signal })
    .then(r => r.json()).then(setUser)
    .catch(e => { if (e.name !== 'AbortError') throw e });
  return () => ctrl.abort();
}, [id]);
```

// Cancelled-флаг

```
useEffect(() => {
  let cancelled = false;
  loadAsync().then(d => { if (!cancelled) setData(d); });
  return () => { cancelled = true; };
});
```

```
}, [id]));
```

МОГУТ ДОСПРОСИТЬ

- 01 AbortController vs cancelled-флаг — что когда?
- 02 Что делает react-query под капотом?
- 03 Почему useEffect не должен возвращать promise?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

Как избежать race condition в effect?

ОПИСАНИЕ

Race в effect – старый запрос вернулся после смены deps и затёр свежий результат. Лечение: cancelled-флаг в замыкании cleanup; AbortController с abort в cleanup; сравнение версии запроса с актуальной перед setState. Все три рабочие; AbortController точнее, флаг проще.

КАК РАБОТАЕТ

- 01 Эффект перезапускается при смене dep – старая копия уходит.
- 02 Старый fetch продолжает работать.
- 03 Cleanup ставит cancelled = true / abort().
- 04 Старый then проверяет cancelled или ловит AbortError.
- 05 react-query / SWR используют версионирование запроса под капотом.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Загрузка по id (профиль пользователя).
- 02 Live-search.
- 03 Любой effect с setState после async.

ПОДВОДНЫЕ КАМНИ

- 01 Без защиты – "прыгающие" данные.
- 02 Race появляется только под нагрузкой/медленной сетью.
- 03 Часть UI обновляется, часть нет – баг трудно воспроизвести.

КОД

// Защита через флаг

```
useEffect(() => {
  let cancelled = false;
  fetch(`/u/${id}`).then(r => r.json()).then(d => {
    if (!cancelled) setUser(d);
  });
  return () => { cancelled = true; };
}, [id]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем AbortController лучше cancelled-флага?
- 02 Как воспроизвести race в dev?
- 03 Что делает react-query?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/you-might-not-need-an-effect#fetching-data>

useLayoutEffect**STRICT MODE****IDEMPOTENT****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Слой layout-эффектов и устойчивость к двойному запуску.

ВОПРОСЫ ДНЯ

- Q13 Когда нужен useLayoutEffect?
- Q14 Чем useLayoutEffect отличается от useEffect?
- Q15 Что происходит с effects в Strict Mode?
- Q16 Почему effect может выполняться дважды в dev?
- Q17 Как писать idempotent effects?

ВОПРОС Q13

Когда нужен `useLayoutEffect`?

ОПИСАНИЕ

`useLayoutEffect` нужен, когда логика зависит от геометрии DOM ИЛИ должна выполняться ДО того, как браузер нарисует кадр. Например: измерить ширину и позиционировать `tooltip`; синхронизировать `scroll`; избежать мигания при условном рендере. В отличие от `useEffect`, `useLayoutEffect` синхронный – браузер ждёт его завершения.

КАК РАБОТАЕТ

- 01 `useLayoutEffect` выполняется после `mutation`, до `paint`.
- 02 `useEffect` – после `paint`, асинхронно.
- 03 Тяжёлая работа в `layout-effect` блокирует кадр.
- 04 Идеально: измерить → `setState` → рендер → `paint` (всё в одном кадре).
- 05 Избегать сетевых вызовов и тяжёлых вычислений.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Tooltip` позиционирование.
- 02 Анимации `start-state` (FLIP).
- 03 `Scroll sync`.
- 04 Условный рендер с измерением размеров.

ПОДВОДНЫЕ КАМНИ

- 01 Долгий `useLayoutEffect` = пропущенный кадр.
- 02 Не работает в SSR – нужен `useEffect-fallback`.
- 03 В Strict Mode dev – двойной запуск.
- 04 Для большинства задач достаточно `useEffect`.

КОД

// Замер до `paint`

```
useLayoutEffect(() => {  
  const w = ref.current!.offsetWidth;  
  setMeasured(w);  
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `useLayoutEffect` отличается от `useEffect`?
- 02 Что показать в SSR?
- 03 Почему он опасен в перформансе?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useLayoutEffect>

ВОПРОС Q14

Чем `useLayoutEffect` отличается от `useEffect`?

ОПИСАНИЕ

Время запуска и blocking-семантика. `useEffect` – асинхронный, после `paint`, не блокирует кадр. `useLayoutEffect` – синхронный, после `mutations` и до `paint`, БЛОКИРУЕТ кадр. Поэтому 99% случаев – `useEffect`; `useLayoutEffect` – только для DOM-измерений и anti-flicker сценариев.

КАК РАБОТАЕТ

- 01 Порядок: `mutation` → `useLayoutEffect` (sync) → `paint` → `useEffect` (async).
- 02 `useLayoutEffect` видит обновлённый DOM до браузерного `paint`.
- 03 `useEffect` видит обновлённый DOM, но уже после `paint`.
- 04 Cleanup-таймизация аналогична.
- 05 На SSR `useLayoutEffect` выводит warning – нужно `useEffect-fallback`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `useEffect` – большинство задач.
- 02 `useLayoutEffect` – измерения, синхронизация DOM до `paint`.

ПОДВОДНЫЕ КАМНИ

- 01 Long-running `useLayoutEffect` – отложит `paint`.
- 02 `useLayoutEffect` в SSR – не работает.
- 03 Cargo-cult-замена `useEffect` – потеря производительности.

КОД

// Сравнение

```
// видим мигание при изменении layout – useLayoutEffect
// можно подождать paint – useEffect (по умолчанию)
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда оправдан `useLayoutEffect`?
- 02 Что покажет SSR?
- 03 Чем отличается порядок `cleanup`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useLayoutEffect>

Что происходит с effects в Strict Mode?

ОПИСАНИЕ

В Strict Mode dev React запускает каждый effect ДВАЖДЫ при mount: setup → cleanup → setup. Это специально, чтобы выявлять не-идемпотентные эффекты, забытый cleanup и баги, которые проявятся в concurrent. В production поведение нормальное – один setup.

КАК РАБОТАЕТ

- 01 Mount → effect setup → effect cleanup → effect setup.
- 02 Только в development и только в Strict Mode.
- 03 В концепте: симулирует unmount-remount, который может произойти в concurrent.
- 04 Эффект должен быть идемпотентен (см. Q17).
- 05 Cleanup – обязательно симметричен setup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Dev-окружение.
- 02 Тесты могут показывать двойные вызовы.
- 03 Линтинг и code-quality.

ПОДВОДНЫЕ КАМНИ

- 01 Двойной запрос в dev – пугает, но это «фича».
- 02 Без cleanup – двойные подписки/таймеры.
- 03 Не-идемпотентные effects валятся в Strict Mode.
- 04 Production-поведение – один раз; не маскируйте баги проверкой на dev.

КОД

// Идемпотентный effect

```
useEffect(() => {
  socket.connect();
  return () => socket.disconnect();
}, []); // ОК даже при двойном setup
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему React делает двойной запуск в Strict Mode?
- 02 Что значит идемпотентный effect?
- 03 Как отличить настоящий баг от Strict-симуляции?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/StrictMode>

ВОПРОС Q16

Почему effect может выполняться дважды в dev?

ОПИСАНИЕ

Это поведение Strict Mode – намеренное. React запускает `setup/cleanup/setup`, чтобы код был готов к concurrent-режиму, где компоненты могут размонтироваться и монтироваться без видимого переключения. Если эффект ломается при двойном запуске – он не идемпотентный, и это будущая бага.

КАК РАБОТАЕТ

- 01 React 18 Strict Mode – двойной mount компонента и двойной запуск effect.
- 02 Цель – найти non-idempotent effects до prod.
- 03 В prod – обычное поведение.
- 04 Если в dev двойной effect ломает – переписать с `cleanup` или `ref-pattern`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Все React 18+ приложения с Strict Mode (он рекомендован по умолчанию).
- 02 Аудит легаси-эффектов.

ПОДВОДНЫЕ КАМНИ

- 01 Удалять Strict Mode чтобы «починить» – bad smell.
- 02 Двойной POST-запрос в dev – обычно надо убрать запрос из effect (это user action, не sync).
- 03 Logging analytic event – двойной в dev, нормальный в prod.

КОД

// Не-идемпотентный POST

```
// антипаттерн
useEffect(() => {
  fetch('/api/like', { method: 'POST' });
}, []);
// dev отправит дважды – нужно перенести в onClick
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие effects надо переносить из `useEffect` в обработчики?
- 02 Как убрать Strict Mode правильно?
- 03 Что меняется в production?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/StrictMode>

Как писать idempotent effects?

ОПИСАНИЕ

Идемпотентный effect – повторный запуск (после cleanup) приводит к тому же результату. Достигается симметричным setup-teardown: подписался → отписался; setTimeout → clearTimeout; добавил элемент → удалил. Не-идемпотентные (POST-запросы, мутирующие глобал) выносятся в обработчики событий.

КАК РАБОТАЕТ

- 01 Setup создаёт ресурс – cleanup освобождает.
- 02 Эффект НЕ должен иметь персистентных side effects (POST, write to disk без отмены).
- 03 Аналитика mount – допустима, дубль в dev – терпимый.
- 04 Если ресурс глобальный – счётчик ссылок (refcount).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Подписки.
- 02 Таймеры.
- 03 Анимации, инициализация UI-либ.
- 04 WebSocket-подключения.

ПОДВОДНЫЕ КАМНИ

- 01 POST-запрос на mount – не идемпотентен (без UUID-идемпотентности).
- 02 Side effect в module-level (window.x = ...) – не отменяется.
- 03 Refcount не обнулится при незакрытых подписках.

КОД

// Идемпотентность

```
useEffect(() => {
  const sub = stream.subscribe(handler);
  return () => stream.unsubscribe(sub);
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие effects не могут быть идемпотентными?
- 02 Что делать с POST-запросом на mount?
- 03 Как реализовать refcount для глобального ресурса?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/learn/synchronizing-with-effects>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

SUBSCRIPTIONS**TIMERS****DOM LISTENERS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три классических ресурса в effect — корректно подписаться и снять.

ВОПРОСЫ ДНЯ

Q18 Как работать с subscriptions?

Q19 Как работать с timers?

Q20 Как работать с DOM listeners?

Как работать с subscriptions?

ОПИСАНИЕ

Subscriptions – подписки на события извне (WebSocket, EventEmitter, RxJS, Mongo-changestream). Setup в useEffect: подписаться, сохранить unsubscribe-handle. Cleanup: вызвать unsubscribe. Для интеграции с concurrent – useSyncExternalStore.

КАК РАБОТАЕТ

- 01 Подписка возвращает unsubscribe-функцию (или handle для отписки).
- 02 В cleanup вызвать её.
- 03 Если источник в deps – переподписка при смене.
- 04 Для shared subscriptions – refcount-обёртка (или react-query-стайл).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 WebSocket-каналы.
- 02 EventEmitter / pub-sub.
- 03 RxJS Observables.
- 04 External state stores.

ПОДВОДНЫЕ КАМНИ

- 01 Без unsubscribe – утечка и работа после unmount.
- 02 Двойная подписка в Strict Mode – нужна симметрия.
- 03 Подписка на event с inline-функцией – нет ссылки для снятия (нужно сохранить).

КОД

// WebSocket

```
useEffect(() => {
  const ws = new WebSocket(url);
  ws.onmessage = e => setMsg(JSON.parse(e.data));
  return () => ws.close();
}, [url]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое useSyncExternalStore?
- 02 Как сделать refcount-обёртку?
- 03 Что произойдёт без cleanup?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q19

Как работать с timers?

ОПИСАНИЕ

Timers – `setTimeout/setInterval`. Setup: запустить и сохранить `id`. Cleanup: `clearTimeout / clearInterval`. В `deps` – то, от чего зависит `callback`. При `stale-closure`-проблеме – `ref-pattern` или `functional update setState`.

КАК РАБОТАЕТ

- 01 `id` из `setInterval` сохранять в локальной переменной `effect-scope`.
- 02 Cleanup вызывает `clear`.
- 03 Для интервалов с зависимостью на `state` – перезапускать или `ref-pattern`.
- 04 В `Strict Mode dev` таймер запускается дважды – `cleanup` корректно отменяет первый.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Polling.
- 02 Анимации без `raf`.
- 03 Idle-таймауты (`logout`).
- 04 `Debounce / throttle` (внутри обработчика).

ПОДВОДНЫЕ КАМНИ

- 01 Без `clear` – таймер живёт после `unmount`.
- 02 `Stale-closure` в `setInterval` – без `ref` видит старое `state`.
- 03 `setTimeout` в `effect` без `cleanup` – `fire-and-forget`.

КОД

```
// Polling
```

```
useEffect(() => {  
  const id = setInterval(load, 5000);  
  return () => clearInterval(id);  
}, [load]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `setInterval` опасен в `effect`?
- 02 Какой `ref-pattern` для свежего `state`?
- 03 Как делать `polling` с `retry`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useEffect>

Как работать с DOM listeners?

ОПИСАНИЕ

DOM-listeners на window/document/body вне React-дерева – обычно вешают через `useEffect`. `addEventListener` в `setup`, `removeEventListener` в `cleanup`. Важно: ссылка на handler должна быть стабильной для снятия (одна и та же функция). Для сложных handler'ов – `useEvent-pattern` или `ref`.

КАК РАБОТАЕТ

- 01 `useEffect: window.addEventListener('resize', handler).`
- 02 `Cleanup: window.removeEventListener('resize', handler).`
- 03 Handler передаётся переменной – иначе `removeListener` не найдёт.
- 04 Опции `event-listener` (`passive`, `capture`) – учитывать.
- 05 Для часто меняющегося state – `ref-pattern` (см. Q14 из Hooks).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Resize`, `scroll`.
- 02 `Keyboard shortcuts`.
- 03 `Click-outside` для модалок.
- 04 `Drag-and-drop`.

ПОДВОДНЫЕ КАМНИ

- 01 Inline-функция – каждый раз новая, `removeListener` не сработает.
- 02 Без `cleanup` – двойные listeners в `Strict`.
- 03 Тяжёлый `scroll-handler` без `throttle` – лаги.
- 04 `passive: true` важно для `touch/scroll perf`.

КОД

// Resize

```
useEffect(() => {
  function onResize() { setW(window.innerWidth); }
  window.addEventListener('resize', onResize);
  return () => window.removeEventListener('resize', onResize);
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему inline-handler нельзя?
 - 02 Что такое `passive listener`?
 - 03 Как `throttle handler` корректно?
-

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать роль useEffect и три кейса, когда он не нужен (Q1, Q2)
- ☐ [] Объяснить «синхронизация с внешним миром» (Q3)
- ☐ [] Расписать антипаттерн derived state через effect (Q4)
- ☐ [] Сказать про правила deps и линтер exhaustive-deps (Q5, Q6, Q7, Q8)
- ☐ [] Объяснить infinite loop и правильный cleanup (Q9, Q10)
- ☐ [] Описать AbortController и race в effect (Q11, Q12)
- ☐ [] Развести useEffect и useLayoutEffect (Q13, Q14)
- ☐ [] Сказать про Strict Mode и идемпотентность (Q15, Q16, Q17)
- ☐ [] Расписать subscriptions / timers / DOM listeners в эффекте (Q18, Q19, Q20)